

Schemas, Constraints, and Evidence

Inspect the current structure, then make invalid states harder to store.

Schemas protect meaning at the storage boundary

The best invalid row is the row the database refuses to store.

IDENTITY

Primary and unique keys distinguish logical records.

DOMAIN

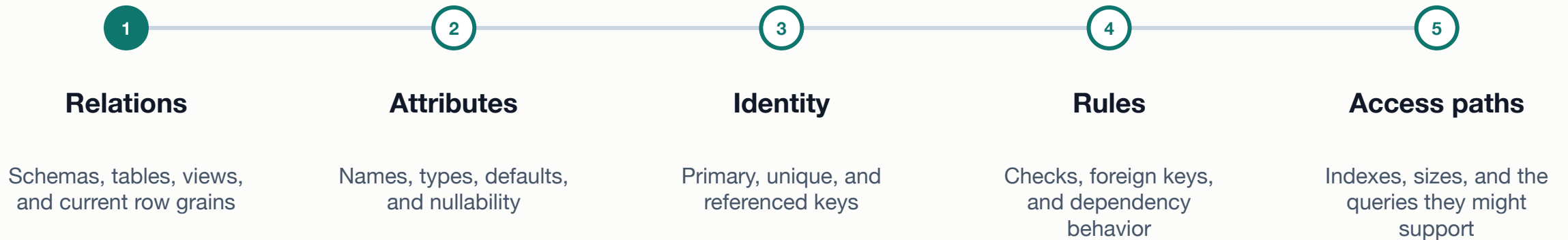
Types, nullability, and checks restrict allowed values.

RELATIONSHIP

Foreign keys protect references between identities.

Perform a schema x-ray before proposing treatment

Inventory the current state so the change has a verified starting point.



PostgreSQL catalogs reveal the structure the server enforces

```
SELECT table_schema, table_name
FROM information_schema.tables
WHERE table_schema = 'metro_support'
ORDER BY table_name;

SELECT column_name, data_type, is_nullable
FROM information_schema.columns
WHERE table_schema = 'metro_support'
  AND table_name = 'tickets'
ORDER BY ordinal_position;
```

Evidence, not decoration

- Confirm the database and schema first.
- Preserve column order for a readable x-ray.
- Pair metadata with the business rule it serves.

Different constraints reject different bad states

Row identity

PRIMARY KEY rejects duplicate and null identity. UNIQUE protects another candidate identity.

Test duplicate values.

Value domain

NOT NULL rejects absence. CHECK rejects values or combinations outside an explicit predicate.

Test a boundary value.

Relationship

FOREIGN KEY rejects a child reference when no permitted parent identity exists.

Test an orphan.

Lab 1: SQL clinic and schema x-ray

Retrieve core SQL, inspect the current Metro Support schema, and identify one integrity risk.

- Complete the cumulative query checkpoint and state result grain.
- Inventory tables, ticket columns, keys, constraints, and indexes.
- Name one bad state the current schema permits or one rule already protected.

SUBMIT ONE THING / one schema x-ray with SQL evidence

Make the rule observable through success and failure

A constraint definition is a claim. Test behavior before trusting the claim.

Translate business rules into testable database behavior

Business rule	Mechanism	Expected success	Expected failure
Every ticket has a stable identity	PRIMARY KEY	new unique ticket_id	duplicate or null ticket_id
Status uses the supported vocabulary	CHECK	status = 'open'	status = 'almost_done'
Requester must already exist	FOREIGN KEY	known user_id	unknown requester_id
External reference is not reused	UNIQUE	new reference	second matching reference

Create the rule, then challenge it inside a transaction

```
ALTER TABLE metro_support.tickets
ADD CONSTRAINT tickets_status_check
CHECK (status IN
      ('new','open','in_progress','resolved','closed'));

BEGIN;
INSERT INTO metro_support.tickets (...)
VALUES (... , 'almost_done', ...);
-- Expected: check-constraint violation
ROLLBACK;
```

Interpret the outcome

- Name the constraint for useful errors.
- Use a deliberately invalid synthetic row.
- Rollback keeps the fixture reusable.

An index is an access path, not another integrity rule

Constraint

- Defines a valid database state
- Rejects a violating write
- Supports correctness claims within its scope

Index

- Provides an ordered or specialized access path
- May reduce work for matching query shapes
- Consumes storage and write/maintenance work

A constraint evidence record has four parts

- 1 State the invalid state in plain language and name the database mechanism.

- 2 Show the definition and the environment where it is installed.

- 3 Run one valid boundary case and one expected-invalid case.

- 4 Record the observed behavior, rollback, and one rule still outside the constraint.

Lab 2: Build and test one integrity rule

Turn one Metro Support business rule into a database constraint with positive and negative evidence.

- Choose one rule and predict the valid and invalid test outcomes.
- Create the constraint with a clear name in the disposable course schema.
- Run one expected success and one expected failure, then roll back and interpret both.

SUBMIT ONE THING / one constraint evidence record

Inspect first; constrain second; verify behavior third

A schema is trustworthy when its promises can be observed at the boundary it controls.

- Which invalid state does the rule reject?

- Which metadata proves the rule exists?

- What valid and invalid tests establish its boundary?